

```
Mazda RX4 21.0 6 160 110 3.90 2.620 16.46 0 1 4 4 1970
Mazda RX4 Wag 21.0 6 160 110 3.90 2.875 17.02 0 1 4 4 1970
Datsun 710 22.8 4 108 93 3.85 2.320 18.61 1 1 4 1 1973
Hornet 4 Drive 21.4 6 258 110 3.08 3.215 19.44 1 0 3 1 1974
Hornet Sportabout 18.7 8 360 175 3.15 3.440 17.02 0 0 3 2 1977
Valiant 18.1 6 225 105 2.76 3.460 20.22 1 0 3 1 1962
Volvo 142E 21.4 4 121 109 4.11 2.780 18.60 1 1 4 2 1966
```

There also exists the *merge()* function that combines two data sets. In the following example, the 'data' and 'new_mtcars' data sets are merged and sorted on the *mpg* column in descending order. The *merge* function uses common variables for its operation.

```
> merge(data, new_mtcars)
  mpg cyl disp  hp drat   wt  qsec vs am gear carb year
1 18.1   6  225 105 2.76 3.460 20.22  1  0   3   1 1962
2 18.7   8  360 175 3.15 3.440 17.02  0  0   3   2 1977
3 21.0   6  160 110 3.90 2.620 16.46  0  1   4   4 1970
4 21.0   6  160 110 3.90 2.875 17.02  0  1   4   4 1970
5 21.4   6  258 110 3.08 3.215 19.44  1  0   3   1 1974
6 22.8   4  108  93 3.85 2.320 18.61  1  1   4   1 1973
```

The *merge()* function allows the following arguments:

- *x*: A data frame to combine with.
- *y*: Another data frame to combine with.
- *by*: The vector of column names for merge.
- *by.x*: The column names in *x* to be used for combining the data.
- *by.y*: The column names in *y* to be used for the merge operation.
- *sort*: A Boolean value on whether to sort the results or not.
- *incomparables*: A list of variables that cannot be compared.

The *intersect()* function returns the common variables between two vectors. For example:

```
> intersect(data, new_mtcars)
  mpg cyl disp  hp drat   wt  qsec vs am gear carb
Mazda RX4 21.0 6 160 110 3.90 2.620 16.46 0 1 4 4
Mazda RX4 Wag 21.0 6 160 110 3.90 2.875 17.02 0 1 4 4
Datsun 710 22.8 4 108 93 3.85 2.320 18.61 1 1 4 1
Hornet 4 Drive 21.4 6 258 110 3.08 3.215 19.44 1 0 3 1
Hornet Sportabout 18.7 8 360 175 3.15 3.440 17.02 0 0 3 2
Valiant 18.1 6 225 105 2.76 3.460 20.22 1 0 3 1
```

You can combine vectors or data frames into a single data frame using the *make.groups()* function, as shown below:

```
> make.groups(new_mtcars$gear, new_mtcars$cyl)
      data      which
new_mtcars$gear1 4 new_mtcars$gear
new_mtcars$gear2 4 new_mtcars$gear
```

```
new_mtcars$gear3 4 new_mtcars$gear
new_mtcars$gear4 3 new_mtcars$gear
new_mtcars$gear5 3 new_mtcars$gear
new_mtcars$gear6 3 new_mtcars$gear
new_mtcars$cyl1 6 new_mtcars$cyl
new_mtcars$cyl2 6 new_mtcars$cyl
new_mtcars$cyl3 4 new_mtcars$cyl
new_mtcars$cyl4 6 new_mtcars$cyl
new_mtcars$cyl5 8 new_mtcars$cyl
new_mtcars$cyl6 6 new_mtcars$cyl
```

Transform

Computation can be performed to update existing values or add new data. For example, the *qsec* column in the *mtcars* data set represents the time in seconds to reach a quarter mile. You can convert the data into minutes using the following calculation:

```
> data$minsec <- (data$qsec / 60)
> data
      mpg cyl disp  hp drat   wt  qsec vs am gear carb minsec
Mazda RX4 21.0 6 160 110 3.90 2.620 16.46 0 1 4 4 0.2743333
Mazda RX4 Wag 21.0 6 160 110 3.90 2.875 17.02 0 1 4 4 0.2836667
Datsun 710 22.8 4 108 93 3.85 2.320 18.61 1 1 4 1 0.3101667
Hornet 4 Drive 21.4 6 258 110 3.08 3.215 19.44 1 0 3 1 0.3240000
Hornet Sportabout 18.7 8 360 175 3.15 3.440 17.02 0 0 3 2 0.2836667
Valiant 18.1 6 225 105 2.76 3.460 20.22 1 0 3 1 0.3370000
```

The *transform()* function also exists to change variables in a data frame. You can update multiple columns with the *transform()* function, as shown below:

```
> transform(new_mtcars, wt=wt*0.4545, year=2021-year)
      mpg cyl disp  hp drat   wt  qsec vs am gear carb year
Mazda RX4 21.0 6 160 110 3.90 1.190790 16.46 0 1 4 4 51
Mazda RX4 Wag 21.0 6 160 110 3.90 1.306688 17.02 0 1 4 4 51
Datsun 710 22.8 4 108 93 3.85 1.054440 18.61 1 1 4 1 48
Hornet 4 Drive 21.4 6 258 110 3.08 1.461218 19.44 1 0 3 1 47
Hornet Sportabout 18.7 8 360 175 3.15 1.563480 17.02 0 0 3 2 44
Valiant 18.1 6 225 105 2.76 1.572570 20.22 1 0 3 1 59
```

The *apply()* function can be used on an array or matrix. It takes three arguments — an array, dimensions, and a function. The function is applied to the specific dimensions of the array or matrix. In the following example, the rows (MARGIN=1) represent the first dimension while the columns (MARGIN=2)